

Event-Driven Hierarchical Rule Composition and Saturation

Coarse and Smooth Layers, Dissipative and Infi Rule Compositions

Technical report 22 September 2026

Implementation baseline: 258eef4 Native-figure revision v0.2.2

Abstract

Equality saturation shares concrete expressions, but repeated derivations with different bindings can still generate large graphs and dense execution histories. We present a hierarchy that treats rule applications as evidence for reusable reasoning units. Committed events form a versioned dependency graph. Relative to a selected context, applications are classified as coarse or smooth and organized into coarse layers, smooth layers, and CS units. Sequential CSCS composition preserves dependencies; guarded CCSS regrouping combines the coarse and smooth parts without discarding interface obligations. An explicit *ripen* operation computes local closure under staged external inputs, while exact state comparison preserves types, equality, visibility, and the relevant rule environment. We then define parameterized fractal rule compositions through stable binding and effect transfers, distinguishing well-founded dissipative processes from families with arbitrarily long finite prefixes. These abstractions suggest finite scheduling, endpoint reduction, and demand-driven symbolic feedback to ripen. A Rust/native prototype implements bounded candidate discovery, staged replay, exact comparison of exported local states, resource subscriptions, and promotion of validated compositions. In one fixed experiment it produced nine closed local results, seven distinct representative bodies, and six promoted composition records at depth two. These are isolated symbolic-interface results: no live tier0 replacement, whole-program compression, or acceleration is implemented. General induction and fractal-summary feedback remain proposals.

Keywords: equality saturation; rule composition; local closure; relative binding; event-driven analysis; recursive summaries.

1 Introduction

An e-graph shares e-nodes with identical operators and canonical children, and records equivalence between expressions through union. This eliminates concrete structural duplication. It does not automatically share a derivation such as expansion, commutation, reassociation, and folding across many different bindings. Even when the expression graph shares some nodes, the history of matches and the space of potential rule combinations may continue to grow.

Our objective is to discover reusable reasoning units that can eventually reduce repeated storage and matching. Three objects must remain distinct: a state, a process reaching that state, and the conditions enabling the process. Different processes may reach the same state, while the same rule sequence may behave differently under different bindings and external effects. Neither a list of rule names nor a single endpoint expression is therefore an adequate sharing key.

Egglog combines Datalog-style reasoning with equality saturation and provides the native execution substrate for our prototype [1]. We use *tier0* for ordinary execution, *tier1* for the analysis of applications and their compositions, and *tier2* for binding transfers, recurrence evidence, and symbolic reduction. These

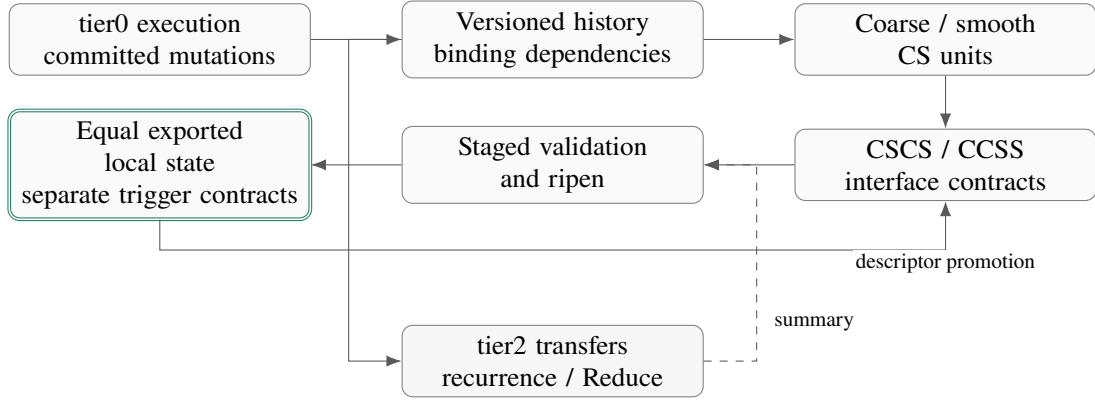


Figure 1: The hierarchy. The dashed connection indicates certified-summary feedback that is not yet automatically integrated into ripen. Finite closure promotion is implemented.

are logical layers: the current tier1 is primarily a Rust data structure rather than a mandatory chain-shaped egglog encoding.

This report gives a bounded implementation account of the hierarchy and audits its evidence boundaries. The implemented contribution is narrower than the surrounding vocabulary: observed applications are grouped into candidate units, candidate compositions are replayed in isolated native cells, and exported local states can be compared and catalogued. The prototype does not replace tier0 execution, discover all useful combinations, prove arbitrary-depth recurrences, or perform runtime storage compression. The diagrams therefore distinguish observed history, replayed local state, and proposed summaries instead of presenting them as one execution pipeline.

1.1 How the mechanisms fit together

Read the finite pipeline in this order: recorded applications supply evidence; CSCS and CCSS propose candidate interfaces; native replay validates those interfaces; ripen computes a local fixed point; exact comparison groups equivalent bodies. These operations work on different objects. Promotion feeds a validated member descriptor back into candidate discovery; it does not use the exported saturated body as an executable macro. Sections 7–9 then develop parameterized families and summary contracts as an extension of this finite pipeline. Their definitions and examples remain useful even where the execution feedback is not yet integrated.

1.2 Reading the running examples

The examples use actual rewrite bodies from `egglog/tests/web-demo/math.egg`. We isolate small rule sets and explicit schedules so that the illustrated dependency is reproducible; this is not a claim that the complete optimizer reaches the same saturated body. In the running Add example, a, b, c abbreviate the concrete eclasses containing `Var("a")`, `Var("b")`, and `Var("c")`. For commutation we alpha-rename its pattern variables to x, y , keeping these formal names distinct from the concrete leaf labels. For the native e-graph pictures we also supply a minimal, standalone program, `examples/math-mini-trace.egg`. It keeps only `Var`, `Add`, associativity and commutation. First inspect the input $a + (b + c)$, then the two orange nodes added by one associativity application, then the complete saturated graph. The topology is exported by egglog; the paper script only annotates it. This gives one concrete thread through the interface, composition and saturation definitions.

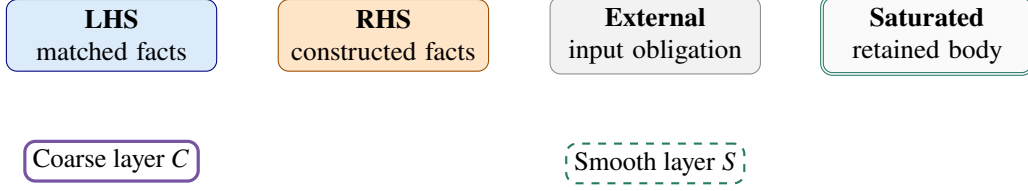


Figure 2: The v0.2.1 palette is retained. In rule diagrams blue/orange mark matched/constructed facts, and purple/teal outlines mark coarse/smooth layers. In native e-graph snapshots blue means present at entry and orange means added since entry; purple/teal dashed clusters identify child/root e-classes, not layers. Each caption specifies which kind of object and edge is being shown.

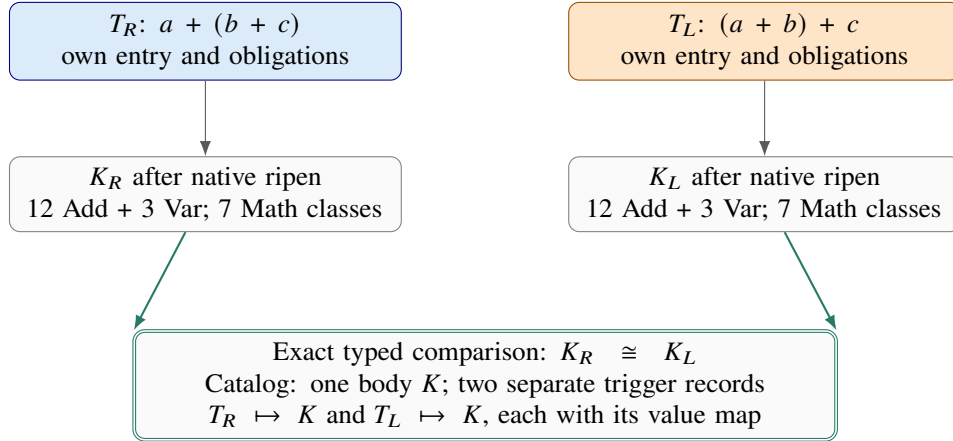


Figure 3: Two different entries can produce the same body. These are catalog and execution records, not e-class boxes. Native replays and exact comparison are checked by the `math_parenthesizations` regression. Equal counts alone are insufficient: the comparison preserves all rows, ports and visibility. The two entries remain distinct, and the live tier0 graph is not replaced.

2 State and Execution Evidence

2.1 Facts, equality, visibility, and updates

We distinguish the components of a runtime state as

$$G = (F, \equiv, V, \Theta, \delta). \quad (1)$$

Here F contains constructor and relation facts, $\equiv$ is equality, V records row visibility, Θ contains function tables and their update semantics, and δ denotes execution and incremental-maintenance state. The supplied rewrite theory is trusted; native replay is not an independent proof of every rule as a mathematical identity. A union is not an ordinary row insertion; subsumption is not simply the absence of a semantic fact; and a function update is not necessarily set union.

For a positive fragment that only adds facts and grows equality, a monotone closure model is useful. General programs require a stronger operational contract. A subsumed row can remain stored while ceasing to participate in matching. Comparing only its contents would lose this distinction.

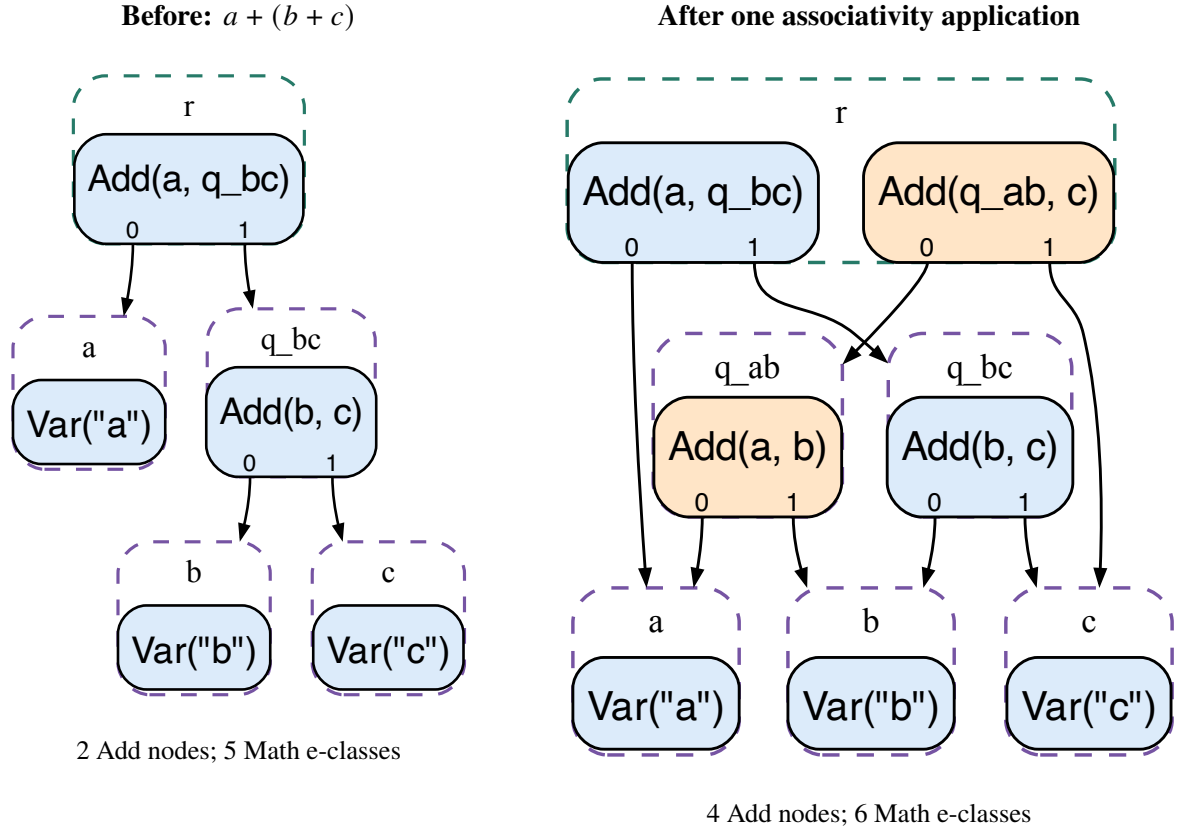


Figure 4: Read the change from left to right. Native egglog introduces $\text{Add}(a, b)$ and $\text{Add}(q_{ab}, c)$ (orange) and unions the latter with the existing root r . Blue nodes were already present. Dashed enclosures are native e-classes; numbered ports are argument positions, and arrows terminate at child classes. Both graphs come from executable prefixes of `math-mini-trace.egg`, through egglog’s own DOT exporter. `Var`’s String payloads are inlined into its label.

2.2 Committed applications rather than matches

An application record is modeled as

$$a_i = (R_i, \beta_i, Q_i, \Delta_i, U_i, W_i), \quad (2)$$

where R_i is a rule, β_i its binding, Q_i the facts and equality evidence read, Δ_i the committed writes, U_i the union information, and W_i update or visibility information. A logical match, a surviving action lane, an inserted row, and a deduplicated write are different events. Our provenance must not silently identify them.

An edge from a_i to a_j records that a_j used a then-valid fact version or equality witness supplied by a_i . Equal values can suggest interaction, but value equality alone does not establish this causal edge. The event graph is acyclic in time even when a rule consumes its own earlier output. Cycles in a quotient graph of recurring patterns instead denote recursion between pattern instances, not cycles in the original event history.

2.3 Relative binding as an interface map

A relative binding maps an input of one unit to an output, a carried input, or a new external port:

$$\rho : \text{In}(B) \longrightarrow \text{Out}(A) \cup \text{In}(A) \cup E. \quad (3)$$

It also carries types, alias constraints, and equality evidence. Such a map may be a permutation, a projection through constructor arguments, a construction of new terms, or a reference to another application's result. The notation $\beta' = \beta + \Delta\beta$ is appropriate only for an actually additive coordinate system; it is not a general definition of relative binding.

Definitions and uses follow a dependency order. A later application may refer to an earlier binding, but a forward reference or a collision between names from different instances is invalid. Numbering variables by first appearance alone is not a complete canonicalization of arbitrary branching graphs.

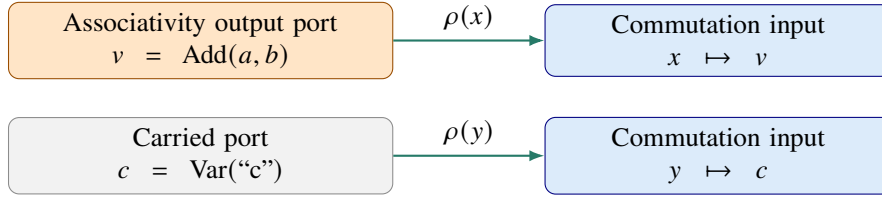


Figure 5: A concrete relative binding from the native Add trace: the next rule matches $\text{Add}(v, c)$, so its formal variables become $x = v$ and $y = c$. This is a port map, not addition of numeric binding offsets. The producer-to-consumer row witness is recorded in the evidence file.

3 Coarse and Smooth Rule Compositions

Let H be a selected local unit. Immediately before its i th stage, let $A_i(H)$ be the evidence supplied by already injected inputs and earlier internal actions. For the chosen recorded witness, define the new external obligation conceptually as

$$E_i(H) = Q_i \setminus A_i(H). \quad (4)$$

The new injection for stage i is not yet included in $A_i(H)$. By convention, the initial triggering input is charged to the first coarse step. This is evidence subtraction, not subtraction of bare IDs: equality, versions, and visibility must be respected.

Definition 1 (Context-relative application kind). *A one-step composition is a coarse rule composition relative to H if it requires a nonempty new external obligation $E_i(H)$. It is a smooth rule composition if all its premises are explained by the current local interface and prior internal results.*

The classification belongs to an application in a chosen combination, not to the rule permanently. A coarse application can become smooth when its suppliers are included in a larger unit. Classification

also depends on the selected witness: a different internal proof may exist without having been discovered. Missing provenance is not proof that no such internal derivation exists.

For the running example, take Add associativity and commutation from `math.egg`. Starting from external $a + (b + c)$, associativity is coarse. Commutation of its produced row $(a + b) + c$, with $x = [a + b]$ and $y = c$, is smooth relative to that two-step unit. Viewed alone, the same commutation requires an external row and is coarse. Figure 6 gives a second concrete case: merely having the denominator binding does not provide its nonzero guard.

A nonempty-premise combination normally begins with a coarse injection. A rule with no premises is a logical exception. An engineering convention such as `Empty -> coarse rule composition` can provide a uniform entry representation, but does not prove that every rule needs external data.

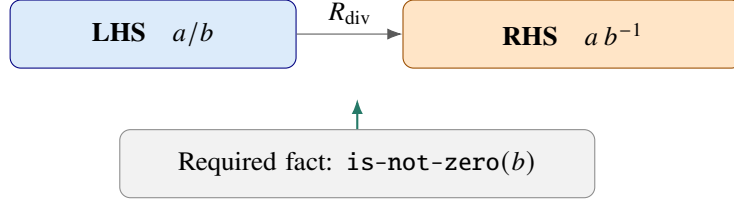


Figure 6: Why a binding is not enough. The guarded division rule is copied from `math.egg`: a native check confirms that it does not apply without `is-not-zero(b)` and applies after that fact is supplied. This illustrates an external obligation; it does not independently establish that the supplied mathematical assertion is true.

4 Layers and CS Units

A *coarse layer* groups coarse applications with explicit binding anchors. A *smooth layer* contains applications whose relevant dependencies can be explained behind that interface. We write a CS unit as

$$U = (C, S, I, O, \Pi), \quad (5)$$

with coarse members C , smooth members S , external interface I , output ports O , and witness information Π .

Figure 7 makes the layer distinction concrete. The native trace contains an associativity write that is subsequently read by commutation with $x = [a + b]$ and $y = c$. A layer encloses applications; its outline must not be confused with the blue/orange roles of facts inside each application.

An intersection is a reason to inspect a pair, not automatically a reason to merge it. For example, merging transitively on the shared constant zero can destroy locality. A common smooth consumer, explicit port connections, candidate budgets, and coverage checks provide stronger selection signals. Requiring a consumer to use outputs from each participating coarse member is a useful heuristic, not a semantic-equivalence theorem.

Units need not form a disjoint partition. A fact can be exposed by several interfaces, and a history can admit multiple useful decompositions. Sharing references and retaining alternatives avoids permanently committing to an early partition.

A *Use* is a concrete occurrence of an installed rule-combination template. It records a template ID, member events, and actual port bindings. This differs from a CS unit, which is defined by contextual obligations. The earlier pipeline admitted only *Uses* to ripen. The current pipeline can create CS candidates even when no template has been installed; *Use* remains an alternative candidate source and an experimental baseline.

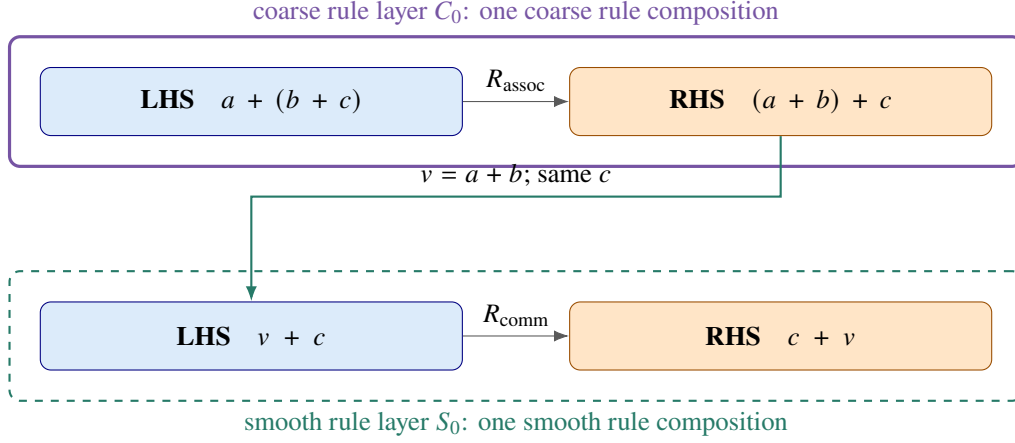


Figure 7: A concrete CS unit from `math.egg`. The first step requires the external entry $\text{Add}(a, \text{Add}(b, c))$; the second reads the associativity output and needs no new external fact. The repeated orange/blue expression is one reused node, shown in two occurrence roles, not duplicated storage. Labels C_0, S_0 describe this selected interface, not globally unique runtime layer IDs.

5 Two Candidate Composition Operators

5.1 Sequential CSCS composition

If U_2 consumes outputs of U_1 , we represent their composition as

$$\text{Seq}(U_1, U_2, \rho). \quad (6)$$

The map ρ separates inputs supplied by U_1 from inputs that still require external injection. It preserves the dependency direction rather than simply concatenating two event arrays. If both units contain applications depending on intermediate results of the other, the complete units cannot be placed in a simple sequential order. A finer decomposition or an unknown result is needed. Dependencies through events outside the proposed pair must also be checked.

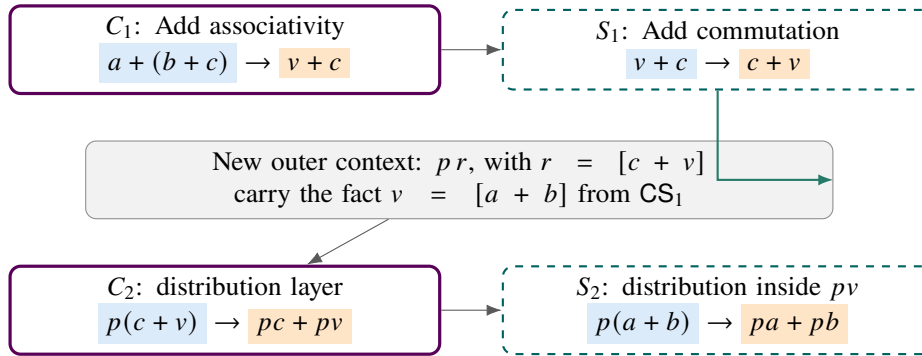


Figure 8: A native-checked CSCS example using associativity, commutation, and distribution from `math.egg`. Distribution may match several retained representatives; one is shown. The declared interface of CS_2 carries the $\text{Add}(a, b)$ fact, so its inner distribution step does not fetch it for free. The fresh outer multiplication context is supplied only after CS_1 .

The multiplication context in Figure 8 is a genuinely new external obligation. The carried $\text{Add}(a, b)$ fact is also explicit: omitting it would incorrectly classify the later inner distribution as self-contained. The diagram shows an interface-level grouping of a native trace; a runtime heuristic may choose a finer layer cut.

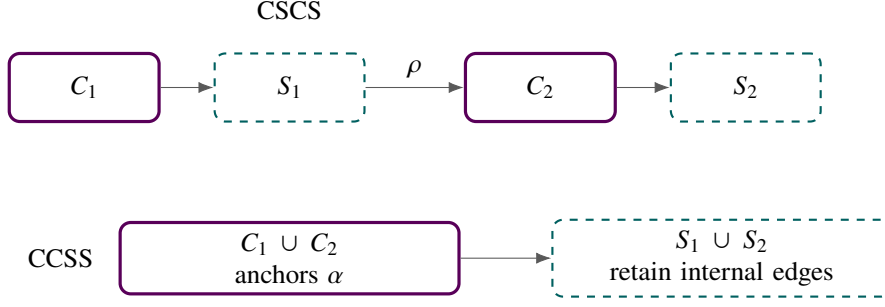


Figure 9: CPCS preserves a cross-unit dependency. CCSS regroupes C and S only under its contract. Neither operation removes external-input obligations.

5.2 Guarded CCSS regrouping

The second operator regroupes the coarse and smooth parts:

$$\text{Par}(U_1, U_2, \alpha) = (C_1 \cup C_2, S_1 \cup S_2, I', O', \Pi'). \quad (7)$$

Here α gives typed anchors between the C parts. The symbol Par names regrouping, not a parallel execution schedule. Dependencies within the combined C or S DAG remain edges.

Our sufficient admission conditions are conservative: disjoint member events, an explicit coarse-port intersection, supported positive constructor/relation/equality operations, and no transitive dependency from any S member to any C member. An exhausted bounded search is unknown, not evidence of independence.

Proposition 1 (Conditional regrouping). *Fix the external-input obligations and a complete dependency witness. If no C member depends on an S member, a topological order exists that places C before S. For supported positive monotone actions, changing between legal internal orders preserves the resulting deductive closure.*

Proof. All predecessors of C lie outside S, so the C subgraph can be topologically ordered before the S subgraph. Every selected premise remains available because internal producers precede consumers and the external obligations are fixed. Independent additions and unions do not retract earlier evidence. Recomputing closure under the same positive theory therefore preserves the consequences. $\square$

This proposition does not move a late external fact earlier. The implementation validates the original staged sequence; its CCSS certificate is conditional on interface obligations, not a claim that the surrounding tier0 execution has been rescheduled. Function updates, subsume, and unsupported guards do not pass the positive-fragment regrouping check.

5.3 Anchors need not be an initial intersection

Suppose one unit supplies $P(a, b)$, another supplies $Q(c, d)$, and a recorded consumer also reads $\text{Link}(b, c)$. The consumer and the Link fact anchor a combination even if the original C parts shared no node. This is a more general interaction witness than a direct coarse intersection. It must not be forced into the particular CCSS admission test.

The prototype stores explicit composition references. After successful saturation, a descriptor retaining the source members can be promoted and become an operand of later candidate compositions. Later jobs flatten member references and replay them; they do not merge the previously exported bodies as execution inputs. Their inherited coarse and smooth descriptions are still conservative: promotion does not automatically find the best reclassification or all legal regroupings.

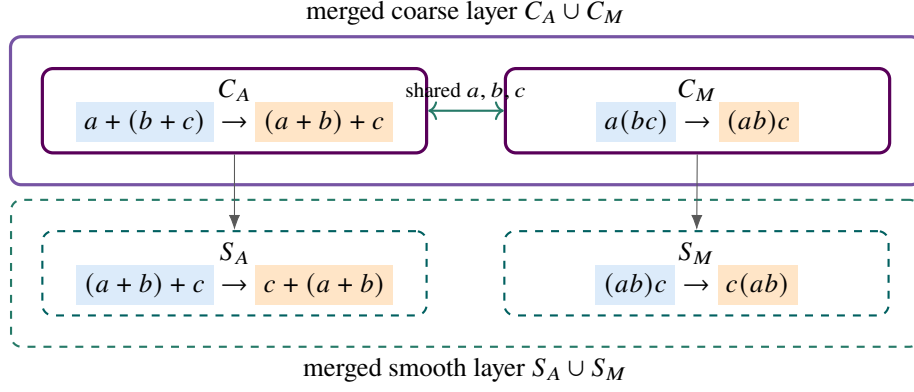


Figure 10: CCSS with real Add/Mul rules from `math.egg`. Both input trees are declared initially and share a, b, c ; there is no S-to-C dependency. The restricted native fragment saturates to 12 Add and 12 Mul rows. Layer outlines indicate regrouping, not deletion of the four original compositions.

6 Ripen and Saturation

6.1 Local closure under staged inputs

Given a rule environment $\mathcal{R}$, a local entry I , and an injection plan J , ripen replays the observed preconditions, injections, and actions, then runs the local rules until a complete sweep makes no update or a budget is reached. The statuses are deliberately distinct:

- **Pending**: not yet processed;
- **Suspended**: insufficient execution budget;
- **Rejected**: entry or validation obligations were not met;
- **Saturated**: an operational fixed point under the specified local model.

A local saturation result neither saturates all of tier0 nor certifies every embedding of the interface. `RipenInput(k)` is an opaque symbolic boundary node: a repeated index preserves identity, while different indices are initially unmerged. It is not a wildcard that matches every value. It does not recover the internal structure of an external eclass. Matches requiring that structure must be treated as additional coarse interaction.

If a second C requires a later external fact, the contract has the form

$$I_1; C_1; S_1; \text{Inject}(I_2); C_2; S_2. \quad (8)$$

Injection declares a caller obligation; it does not derive the fact internally. Promoting I_2 into the initial entry without justification would change the activation conditions.

Definition 2 (Saturated rule composition). *A saturated rule composition consists of a validated composition, its trigger and input mapping, its staged obligations, a saturated realization K , and its certificate. In the current implementation K is a finite materialized body. Cataloguing equal K states does not identify two compositions with different triggers or obligations, and no live matcher reuse is performed. A symbolically saturated infinite body needs the stronger query contract discussed later.*

6.2 Sharing contents without conflating triggers

Different combinations may export identical local states while retaining different triggers, bindings, and injection plans; the current catalog records this equality but does not install shared bodies into tier0. The prototype compares complete exported typed graphs, equality classes, fixed ports, and row visibility. The rule environment must match. When updates or visibility matter, registration order and ruleset scheduling information are also retained.

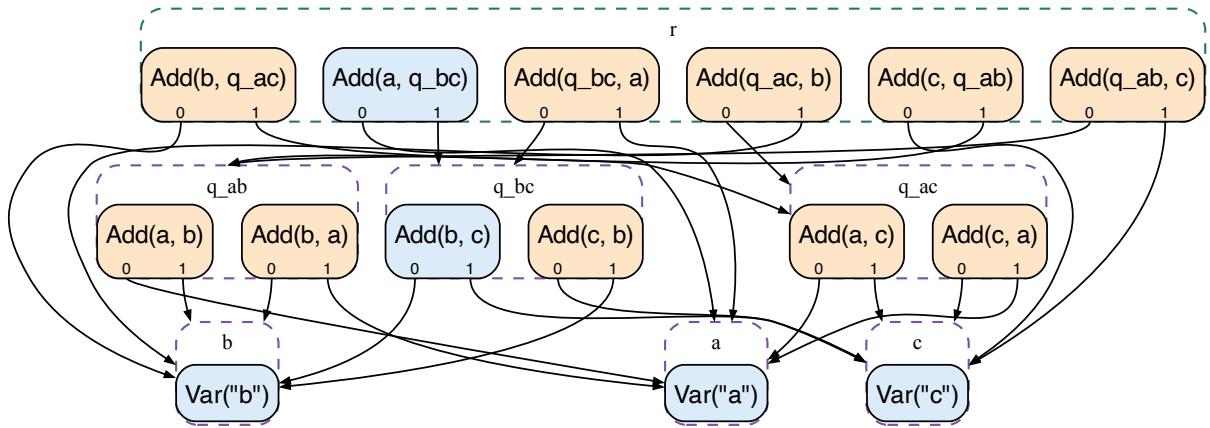


Figure 11: The Add body exported by the `mini.math.egg` fragment. This figure is generated from egglog’s own `-to-dot` serializer: dashed clusters are e-classes, solid boxes are e-nodes, and numbered ports preserve ordered child positions. The paper-side transformation only inlines Var string payloads and annotates the entry-relative colours; it checks that every Math node and ordered child edge from the native DOT survives. The graph has 12 Add nodes, three Var nodes, and seven Math e-classes.

Proposition 2 (Composition of monotone closures). *If Cl is extensive, monotone, and idempotent over a fixed universe, then*

$$Cl(Cl(A) \cup Cl(B)) = Cl(A \cup B). \quad (9)$$

Proof. Extensiveness gives $A \cup B \subseteq Cl(A) \cup Cl(B)$, yielding one inclusion by monotonicity. Conversely, both $Cl(A)$ and $Cl(B)$ are contained in $Cl(A \cup B)$. Monotonicity and idempotence give the reverse inclusion. $\square$

This identity motivates a possible future algorithm starting from existing closures and processing new interaction. The current pipeline instead reconstructs and replays members; it does not implement this identity as a body-composition algorithm. Their union need not already be saturated. The identity supplies no execution budget and is not a theorem about every egglog operation. In particular, subsume changes visibility, and a min/max merge alone does not prove that every surrounding query is monotone.

Exported-content equality is also not equivalence of arbitrary suspended incremental engines: the key does not serialize all of δ . Direct reuse of native matcher or join state is not implemented. The current guarantee is sharing of declared local contents with separate trigger contracts.

6.3 Versioned evidence and recursive promotion

Historical saturated bodies are immutable; the applicability of a concrete instance is separately versioned. Token, table, and scope subscriptions are notified by committed changes. Union joins subscriptions so later notifications follow aliases. Updates and subsumption expire affected evidence. Unknown row ownership causes a conservative table- or scope-level invalidation.

A stale descriptor can propose another symbolic candidate, but cannot bypass native staged validation. Certificates retain `live_substitution=false`: a lease whose version has not changed is not a proof of a current whole-graph embedding. Successful saturated compositions are promoted as new descriptors and registered for further pairing, without replacing tier0 facts.



Same retained equality does not imply the same matching visibility.

Figure 12: The constant-fold and prune rules of `math.egg`. Native execution retains the Add row after subsumption but marks it hidden. An exported local state must record this flag; an instance lease depending on its visibility needs rechecking.

7 Parameterized Rule Compositions

7.1 Stable binding and effect transfers

Repeated rule names are weak evidence. Two ABAB sequences can have different bindings; conversely, independently scheduled branches can represent the same dependency structure despite different event orders. We model a repeated step as

$$z_{k+1} = T(z_k), \quad z_k = (\beta_k, \mathcal{E}_k, \mathcal{P}_k), \quad (10)$$

where β_k is the binding, $\mathcal{E}_k$ accumulated effects, and $\mathcal{P}_k$ the frontier of positions and obligations. In an operational fragment, the effect component must retain structured updates and visibility, not merely a set of inserted facts.

A fractal rule composition is a parameterized family with a trigger, guard, stable transfer, effect contract, and observable interface. “Fractal” denotes recursive structural regularity here, not a claimed geometric dimension. A useful abstract form is

$$\mathcal{F} = (\text{Trigger}, D, T, \text{Guard}, \text{Effects}, \text{Queries}). \quad (11)$$

Finite histories propose such an object; they do not prove that it remains valid for arbitrary counts. The current transfer analysis records Carry, Return, and External port origins and finite witnesses. General induction remains a separate proof obligation.



Construct, construct, carry: same transfer at every depth.

Figure 13: The binding transfer of the integration-by-parts rule in `math.egg`. Growing expression depth does not require a new transfer template: the same Diff/Integral constructors and carried integration variable define each step.

7.2 Trigger states and nonuniform prefixes

A stable phase may begin only after several injections and effect-producing steps:

$$\text{Trigger}(I, \beta_0); T^k; \text{Exit}. \quad (12)$$

Different bindings may require different triggers. An initial “dirty” prefix must not be omitted merely because the later transfer is regular.

For example, swapping Add arguments twice is an identity on argument positions, but the first swap may insert a node and perform a union. For a fixed position and fixed arguments, once both orders and their

equivalence are present, further swaps add no local effect. The stabilization point depends on the initial graph; there is no universal rule equating two swaps with an empty execution.

Nor does a change of rule name in the log establish a turning point. Semi-naive scheduling and other work can interrupt a still-enabled family. A turning point must be defined by a guard, resource, interface, or effect change. Fractal operators need not commute or be orthogonal.

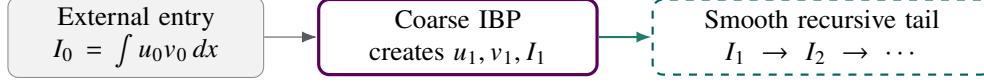


Figure 14: A one-step trigger for the selected IBP family. The first application acquires an external entry; subsequent residual applications reuse constructed facts. Other families can have longer, nonuniform effect-producing prefixes.

7.3 Multiple application positions

For a branching family, use an indexed position domain rather than flattening all steps into a sequence. A schematic binary generator can use

$$D_{k+1} = \{2i, 2i + 1 \mid i \in D_k\}. \quad (13)$$

The text $A \rightarrow B + B$ alone does not prove physical doubling: identical B nodes can be hashconsed. Exponential e-graph growth additionally requires distinct bindings or positions that are not absorbed by existing equality. Three-way branching generally suggests 3^k , not cubic growth in k .

We must therefore distinguish unfolded tree size, parameterized description size, event count, and deduplicated e-node count. A compact binding generator can represent a large family even when the resulting graph does not compress by the same factor.

8 Dissipative and Infi Rule Compositions

8.1 Dissipation is a property of obligations

Definition 3 (Dissipative Rule Composition). *Under a fixed external-supply contract, a fractal process is dissipative if a well-founded measure strictly decreases on every valid recursive step. A simple case is $\mu(z_k) \in \mathbb{N}$ with $\mu(z_{k+1}) < \mu(z_k)$.*

The measure applies to progress steps of the represented process; redundant re-execution of an already accounted match is not a new recursive step. The decreasing quantity may count remaining positions, an unprocessed index range, or outstanding obligations. It does not necessarily count facts being deleted from the e-graph. Facts are generally reusable, so a need for external facts does not itself imply dissipation. Replenishing the external supply may invalidate a phase-local ranking argument.

For a finite sum,

$$(i, s) \mapsto (i + 1, s + x_i), \quad 0 \leq i < n, \quad (14)$$

$\mu = n - i$ proves termination. In reducing five ones to five, it is the remaining reduction positions that decrease. The constant node one can remain a single shared node throughout, and need not disappear at all.

A branching process can also be dissipative while its levels become wider. Suppose each task of rank $r > 0$ creates at most d tasks of rank at most $r - 1$, and rank-zero tasks terminate. On a finite frontier, use

$$\mu = \sum_{p \in \mathcal{P}} (d + 1)^{r(p)}. \quad (15)$$

Replacing a positive-rank task adds at most $d(d+1)^{r-1}$, less than the removed weight $(d+1)^r$. Thus bounded-depth binary expansion can have exponentially wide levels and still terminate. The removed objects in this argument are pending tasks, not retained e-graph facts.

Repeated integration by parts offers another candidate: polynomial degree may decrease if the correct factor and variable are repeatedly differentiated. This needs a polynomial and differentiation contract; finite examples do not prove that arbitrary integral rewrites terminate. We do not report such a general proof as an implemented capability.

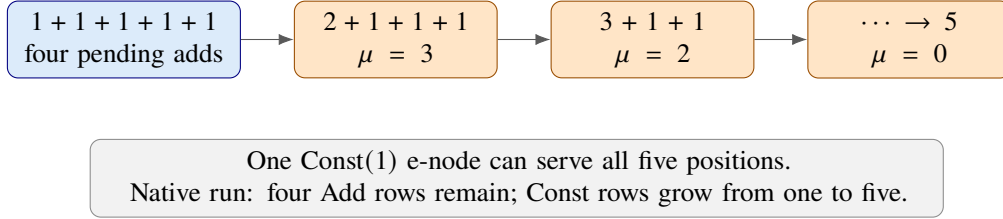


Figure 15: A dissipative composition using the native constant-fold rule from `math.egg`. The initial expression is left-associated; outer Add facts are part of the declared entry. The remaining work frontier decreases, not the stored fact set. No pruning is enabled in this example.

8.2 Infi rule composition as an unbounded prefix interface

Definition 4 (Infinite-prefix family). *For a fixed admitted trigger instance, an infi rule composition admits, under its invariant, a valid prefix of length k for every natural number k . Claiming unbounded growth of the physical graph further requires a freshness argument for facts or positions.*

The initial instance is fixed: choosing a larger terminating input separately for each k does not establish this property. This definition concerns a family of finite prefixes. With nondeterministic choices, arbitrarily long paths do not by themselves establish that one selected execution continues forever. A fixed transfer and consistent prefixes provide a stronger contract.

We distinguish stabilization of effects, bounded periodic representations, and genuinely unbounded generation. Pure smooth execution does not imply the last case, and budget exhaustion proves none of them. Unknown remains an essential outcome.

For example, mathematical-natural-number payloads can describe $A(k, i) \rightarrow B(k, 2i)$, $B(k, 2i + 1)$ and $B(k, i) \rightarrow A(k + 1, i)$. Machine i64 arithmetic additionally requires overflow guards or bounded depth; it cannot inherit an unbounded mathematical conclusion without qualification.

8.3 A productive example from `math.egg`

Let $D_x u$ denote `Diff(x, u)`, let $A_x v$ denote `Integral(v, x)`, and define $I_k = \text{Integral}(u_k v_k, x)$. The actual integration-by-parts rewrite is:

```
(rewrite (Integral (Mul a b) x)
  (Sub (Mul a (Integral b x))
    (Integral (Mul (Diff x a) (Integral b x)) x)))
```

It gives the uniform transfer

$$u_{k+1} = D_x u_k, \quad v_{k+1} = A_x v_k, \quad I_k \Rightarrow u_k v_{k+1} - I_{k+1}. \quad (16)$$

In the IBP-only fragment, with distinct symbolic Var leaves and no additional equalities, the derivative and antiderivative arguments grow structurally. The residual is again an eligible `Integral-of-Mul` node. The two constructors are never simplified by another rule, so this supplies arbitrarily long finite prefixes

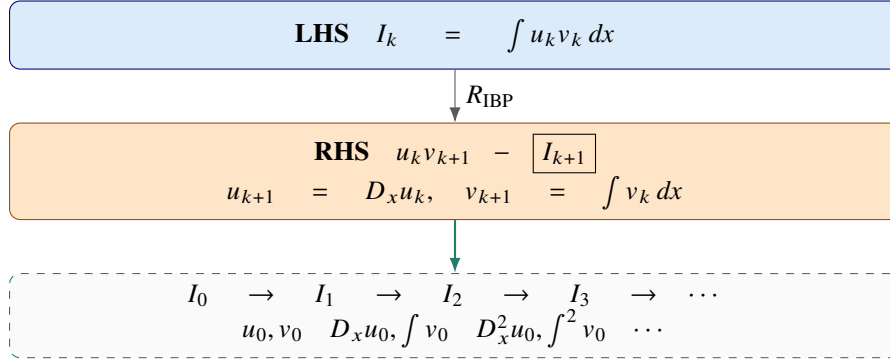


Figure 16: A productive infi rule composition from the actual IBP rule. In the *IBP-only* fragment, the boxed residual regenerates the same pattern with strictly deeper Diff/Integral arguments. Four native steps yield 11, 17, 23, and 29 Math e-nodes. The all-depth argument requires the restricted theory; full `math.egg` may simplify the derivative and terminate this family.

with fresh nodes. No numeric loop counter is evaluated in the graph. In this fragment, each step creates one Diff row, one antiderivative Integral, one residual Integral, two Mul rows, and one Sub row. Distinct derivative and antiderivative arguments prevent their keys from coalescing; IBP unions the integral result with its Sub representative, not with either argument. Thus the constructor-row count is $N_{\text{Math}}(k) = 5 + 6k$, excluding primitive String values and implementation-only global bindings. A native check of prefixes $k = 0, 1, 2, 3, 4$ gives 5, 11, 17, 23, 29 Math e-nodes. These finite measurements check the illustration; the all-depth argument depends on the restricted theory. In full `math.egg`, derivative simplification, other integration rules, and pruning can stop or change the recurrence. With a polynomial derivative contract, the same IBP pattern can instead become dissipative.

8.4 A recursive DSL that becomes an array contract

The previous examples use rules copied from `math.egg`. The following four small programs are deliberately written as a new recursive DSL. They use only a cursor, an upper bound, and a ternary expansion; no array constructor is present in the source. Bake observes the native traces, extracts the repeated binding/effect transfer, and then applies its two structural array-law checks. The generated library is therefore a useful test of the abstraction boundary: the array appears after recognition, not as a hand-written replacement rule.

Listing 1: The two source families supplied to Bake

```
; scan-short.egg and scan-long.egg
(datatype Math (Cursor i64 i64) ...)
(rule ((= node (Cursor i end)) (< i end))
      ((Cursor (+ i 1) end)) :name "scan-next")

; tree-one.egg and tree-four.egg
(datatype Math (Expand i64) (Emit i64))
(rule ((= node (Expand n)) (> n 0))
      ((Emit (+ (* n 3) 0))
       (Emit (+ (* n 3) 1))
       (Emit (+ (* n 3) 2))) :name "split-three")
(rule ((= node (Emit n)) ((Expand n)) :name "resume")
```

The manifest contains two samples for each family. Running `cargorun--offline--bakedocs/papers/examples/recursive-array/manifest.jsonout/paper-recursive-array-v1` produces two portable fractal templates: `fractal_0000` is a `linear_extension` with a `bounded_unit_stride` law; `fractal_0001` is a `recursive_dag` with a `radix_frontier` law of radix three. The identifiers are local to this library; the law kinds and contracts are the meaningful results.

Observed family	Recognized law	Array query produced by Bake
Cursor(i, end)	unit stride; measure $end - i$ decreases	RampArray($end - start$); start = 3, limit = 11 gives values 4, ..., 11 and sum 60
Expand(n) and Emit(n)	radix frontier; residues 0, 1, 2 cover the next level	depth 4, start = 4 gives $3^4 = 81$ values 324, ..., 404 and sum 29484

Table 1: A complete path from a recursive rule DSL to a first-class symbolic array query. Both Bake queries report zero tier0 rule applications, zero element queries, and zero expanded prefixes. The query is over the certified value projection, not over all intermediate facts, unions, or effects.

The first family is dissipative because the remaining distance to the limit strictly decreases. The second family has finite prefixes whose frontier grows by a factor of three; its finite array query is valid only when the requested frontier fits the checked i64 domain. The code checks the guard, constructor shape, residue coverage, return rule, and source schema before emitting either law. It does not infer an array law from matching counts alone.

9 Feedback from Fractal Compositions to Ripen

9.1 Finite scheduling and reduction

A proved dissipative family supplies a principled local count or budget. Ripen can execute its trigger, finite repeat, and exit as a unit, rechecking the contract when external supply or binding changes. A ranking for one family does not prove that all of ripen terminates: another family may reactivate it.

The existing tier2 DSL includes an explicit Reduce node and exact endpoint laws. For a natural count k and a fixed per-step contribution c ,

$$\text{Reduce}(\text{AddConstant}(c), k, t) = t + kc. \quad (17)$$

For contributions $a + id$, $i = 0, \dots, k - 1$, the accumulated contribution is

$$ka + \frac{k(k-1)}{2}d. \quad (18)$$

These are laws for a supplied accumulation contract, not an algorithm discovering that contract from arbitrary history. They concern exact algebra; floating-point reassociation and machine overflow require additional justification.

9.2 A worked endpoint and intermediate-fact example

Consider the guarded recurrence

$$R : f(t) \Rightarrow f(t-1) + 1, \quad t > 0. \quad (19)$$

For natural n and $0 \leq k \leq n$, induction yields the endpoint $f(n-k) + k$. Setting $k = n - m$ gives

$$f(n) \Rightarrow f(m) + n - m, \quad 0 \leq m \leq n. \quad (20)$$

The guard provides a dissipative measure. Removing it over mathematical integers instead permits an unbounded descending sequence, subject to a freshness and effect argument; the same machine-integer implementation is still bounded by its numeric semantics.

The endpoint does not contain every intermediate fact. Another rule may need $f(n-j)$ at a particular j . A reusable summary therefore needs a query interface for $0 \leq j \leq k$, together with the corresponding

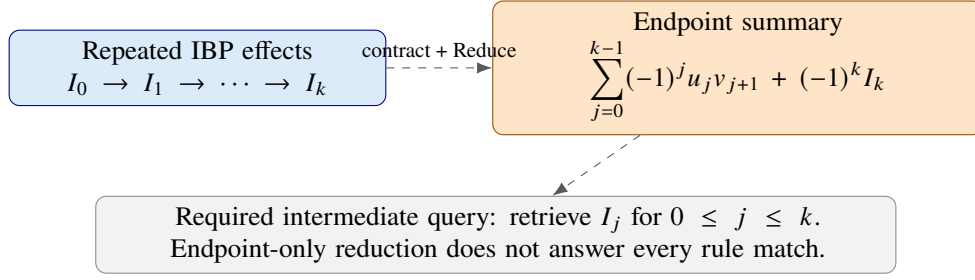


Figure 17: How an infi composition could feed ripen: summarize the alternating endpoint, but retain an indexed residual interface. This is a proposed contract for the worked IBP family with additional exact additive laws, not a claim that the current Reduce DSL automatically discovers or executes this alternating-sum summary.

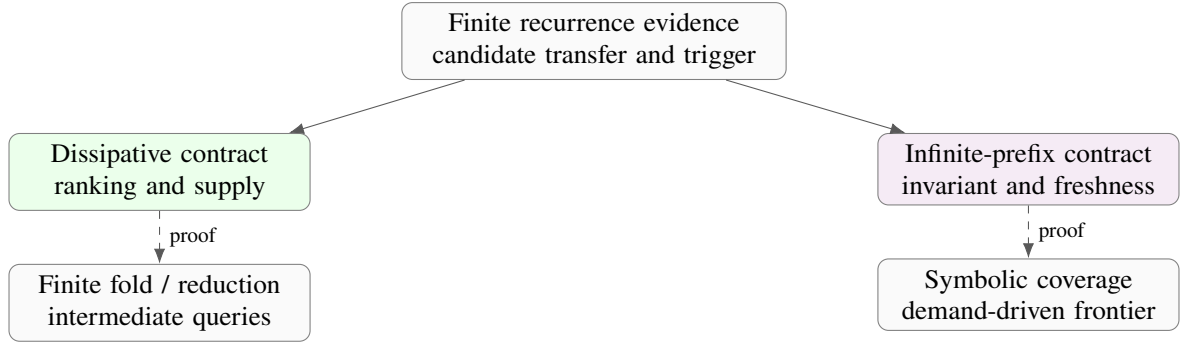


Figure 18: Proof obligations separate observed regularity from summary-based execution. A finite witness is not a certificate of either termination or unbounded generation.

equality/effect information. Replacing the entire history with only the last expression can suppress such interactions and is not a sound implementation of general ripen.

For the IBP example, the actual unrolling is nested Sub nodes. Turning that nesting into the alternating-sum endpoint requires additional exact additive laws. The IBP-only native trace is not claimed to perform this normalization.

9.3 Arrays and symbolic coverage

A useful summary contract exposes a trigger, iteration domain, binding transfer, effects, endpoint queries, and intermediate-position queries. Finite domains naturally give fold and array abstractions; infinite-prefix families require lazy instantiation. Existing logical ArrayGet, Tabulate, Map, and Fold constructs provide relevant vocabulary, but do not by themselves implement mutable memory, alias analysis, cache placement, or GPU scheduling.

An infi rule composition that continually generates fresh facts generally has no finite materialized saturated body. We propose a distinct `SymbolicallyCovered` status: within a specified observation interface, internal matches and effects are represented by a summary and required positions can be produced on demand. This needs both sound answers and coverage of relevant internal matches. If a query cannot be answered, ripen must retain a frontier, expand the requested position, or return unknown. It must not label budget-limited sampling as closure.

9.4 Bake and the present integration boundary

Bake can aggregate observed families from multiple examples into a frozen library. A new example matches the library under explicit binding/effect contracts rather than silently assuming that training examples proved

generality. The local implementation already includes narrow, name-independent proof schemas for a guarded unit-stride scalar recurrence and a bounded radix-frontier payload query [2]. The former preserves the limit and other fields; the latter checks residue coverage and machine-range bounds. Their certified queries do not materialize all tier0 effects.

These narrow queries are useful evidence that executable summaries are possible. They are not a complete fractal rule composition-to-ripen integration. The current native ripen path still reports zero fractal summaries used. General induction and a complete internal-match interface remain open.

Listing 2: Target feedback protocol. The certified-summary branches are not yet generally integrated.

```
on committed event e:
  notify subscribers of changed resources
  update witnessed CS interfaces
  propose bounded CSCS and guarded CCSS candidates

for candidate U within budget:
  validate staged external obligations and bindings
  if a verified dissipative summary applies:
    guide ripen using its finite domain and effect contract
  else if a verified infinite summary covers required queries:
    retain SymbolicallyCovered and a demand-driven frontier
  else:
    run ordinary bounded ripen
  catalogue equal exported states, retaining trigger obligations
  promote certified results and notify subscribers
```

10 Implementation and Evidence

10.1 What is implemented

The baseline is local commit 258eef4. Table 2 separates implemented mechanisms from the proposed summary protocol.

Current bounds include 256 CS versions, 16 observed applications per composite, 64 proposals per composition kind, eight indexed alternatives per token, and 32 partners per awakened unit. Dependency traversal has a separate bound. Unknown ownership can cause broad invalidation. Unsupported nested injections, missing pre-update table witnesses, multi-datatype programs, and some scalar interfaces remain explicit limitations.

10.2 Implementation audit: what the code actually establishes

The implementation boundary is narrower than the terminology used by the diagrams. `native_saturated_rule_composition_pipeline` creates bounded jobs for Uses, dependency candidates, CSCS, and CCSS, then calls `native_ripen` on an isolated entry program. Every job is replayed and validated again; stale evidence cannot authorize substitution. A successful job exports a `SaturatedRuleComposition` and may promote a composition descriptor in the in-memory candidate store. `saturated_rule_composition::compare` checks a bounded typed graph isomorphism including fixed ports and the subsumed-row mask. It does not compare matcher state, scheduler state, or the surrounding tier0 e-graph.

The storage experiment in `experiments/closed_storage` is a separate Python tuple store. Its 20.2 percent retained-object reduction is not egglog RSS, is not a live kernel replacement, and comes with slower queries and union rebuilds. The native pipeline reports `fractal_summaries_used=0`; the Bake and tier2 schemas export or check narrow symbolic summaries but are not general feedback into ripen. The words *shared body*, *saturated composition*, and *fractal rule* therefore refer to conditional artifacts or proposals unless a passage names the exact native check that supports them.

Component	Implemented scope
Native history	Committed applications, binding evidence, mutation records, replay.
Layers and Uses	Context-relative coarse/smooth structures and template instances.
CS composition	Bounded references, conditional certificates, staged validation, promotion.
Exported-state comparison	Bounded exact comparison of typed rows, ports, equality classes, and row visibility; cataloguing only.
Evidence leases	Token/table/scope subscriptions, union aliases, invalidation.
Recurrence analysis	Finite transfer witnesses and selected symbolic endpoint laws.
Bake queries	Narrow checked scalar summaries; no whole-program mutation replacement.
General feedback	Induction and complete symbolic match coverage are not integrated.

Table 2: Implementation scope, based on local source and tests [2, 3].

10.3 Bookkeeping cost and its limits

Let q be changed resources, s subscription hits, and w awakened or newly promoted units. After interfaces have been formed, let L bound interface work, K candidate partners, and B the work of a local dependency check. A conditional bookkeeping estimate is

$$O(q \log N_{\text{res}} + s \log N_{\text{unit}} + w(L + KB)), \quad (21)$$

where the two N values count tracked resources and units. This excludes interface construction, environment checks, serialization, state-isomorphism checks, and the sum of native ripen costs. A dense subscription can still wake many units. Attempt budgets do not bound the worst-case time of one native rule call; the elapsed budget is checked between jobs. These limits prevent a bookkeeping estimate from becoming an unsupported end-to-end complexity claim.

10.4 Recursive promotion and locality

The fixed recursive-CS experiment contains three source chains and is processed by bounded isolated ripen jobs. It reports nine closed jobs, seven equal exported bodies, and six promotion records at maximum composition depth two. Its counters include six layer visits, 75 pair checks, 14 resource notifications, and three subscriber wakes [3]. A promotion adds a new composition descriptor to the bounded candidate store; it does not install a macro into tier0 or substitute a shared body during native matching.

This establishes that validated composition records can be fed back into the candidate store. It does not establish compression, acceleration, or reuse by the live egglog matcher. In a separate registry test, union of two resources wakes their two subscribers among 100 independent units. Shared constants, common tables, or unknown mutation ownership can legitimately require more notification work.

10.5 Historical records are not a controlled ablation

Recorded configuration	Saturation results	Shared states
microbench6 / Use only	14	5
microbench6 / ancestor cones	18	7
microbench6 / explicit CS	16	6
Three-CS recursive example	9	7

Table 3: Historical prototype observations. Here microbench6 is `math-microbenchmark.egg`, not the full `math.egg`. These are not all rerun at one commit; the last row also uses a different input and budget.

The first three records use the same named input and nominal attempt budget, but reflect different implementation versions. They document an evolution rather than isolate a causal improvement. In particular, explicit CS composition did not exceed the earlier cone record in shared-state count.

A separate Python storage prototype expanded 100 C3 triggers into 1,200 Add occurrences, then globally deduplicated them to 304 rows. Retained object bytes including its index and union-find decreased from 25,740 to 20,540, about 20.2%, while queries and joins became slower. This was not egglog RSS and did not replace native tables. It illustrates why sharing potential must be measured together with access and maintenance costs.

For a single snapshot, if $N(U_j)$ is the e-node set covered by instance j , coverage must use a union:

$$\text{Coverage} = \frac{|\bigcup_j N(U_j)|}{|N(G)|}. \quad (22)$$

Multiplying instance counts by template sizes overcounts overlap. Net storage must also include dictionaries, bindings, residual facts, indexes, certificates, versions, and any retained raw history. Shared output identities must remain consistent across instances.

11 Limitations and Research Agenda

Finite recurrence evidence is not induction. Frequency and compression ratio do not replace guard preservation, a ranking argument, or query completeness. Likewise, recognizing a dissipative family does not prove that other rules cannot restart it.

Exported-state equality and live instance substitution remain separate. Opaque parameters omit external structure; version tracking prevents unchecked reuse but does not itself validate a current whole-graph embedding. Subsumption and mutable tables require their operational contracts rather than the pure closure identity.

The discovery procedure is intentionally incomplete. Candidate and dependency bounds control work, and its cost depends on changed resources, subscription hits, and bounded pairing rather than on a promise to find every useful subgraph. Source-environment checks and report construction still have costs. Deeper recursive descriptions can also increase maintenance overhead.

The next evaluation should run all variants at one revision, with matched inputs and budgets, separately ablating Use admission, CSCS, CCSS, recursive promotion, subscriptions, and certified fractal summaries. It should measure repeated internal matching, retained memory, interface-query cost, invalidation/revalidation cost, and omitted candidates, including workloads on which the method loses.

This report uses locally available source and bibliography metadata only. It does not provide an exhaustive related-work survey or establish novelty priority over unreviewed work on graph grammars, library learning, or recursive summaries.

12 Conclusion

We describe a bounded analysis pipeline for locality in recorded rule history. Coarse and smooth labels, CSCS/CCSS candidates, staged replay, and exact comparison of exported local states are implemented under explicit budgets and contracts. The result is a catalog of conditional symbolic-interface observations; it is not a replacement execution engine and it does not make equal local states interchangeable inside tier0.

The recurrence and summary material defines proof obligations for future work. The current code records finite transfer evidence and supports two narrow checked query schemas, while the native ripen report records zero fractal summaries used. Proving arbitrary-depth induction, covering all relevant internal matches, and measuring a net runtime or storage benefit remain open.

References

- [1] Y. Zhang, Y. R. Wang, O. Flatt, D. Cao, P. Zucker, E. Rosenthal, Z. Tatlock, and M. Willsey. Better Together: Unifying Datalog and Equality Saturation. *Proceedings of the ACM on Programming Languages*, 7(PLDI), Article 125, 2023. DOI: [10.1145/3591239](https://doi.org/10.1145/3591239). Bibliographic metadata and abstract checked against local `egglog/CITATION.bib`; full text not independently reviewed for this report.
- [2] *fractal_egraph local prototype*, commit 258eef4, 2026. Core mechanisms: `src/native_cs.rs`, `src/closure_contract.rs`, `src/saturated_rule_composition.rs`; summary schemas: `src/native_bake.rs` and `rules/reduce.egg`. The accompanying verification manifest identifies additional source evidence. Implementation evidence, not an external publication.
- [3] Local experiment records and regression tests, 2026. Records under `experiments/closed_storage/`: `closure-contracts-results.json`, `dependency-candidates-math6.json`, `cs-composition-math6.json`, and `results.json`. Reproduction commands appear in Appendix A; additional test references are listed in the verification manifest. Records are scoped observations, not a uniform benchmark.

A Reproduction and Evidence Levels

A.1 Native DOT figures

`examples/math-mini-trace.egg` marks three executable prefixes: `entry`, one associativity round, and saturation. The generator runs each prefix through native egglog with `-to-dot -to-json`. Raw DOT and JSON are retained under `figures/native/`. The transformation inlines primitive String leaves into Var labels, preserves every Math node and ordered child-class edge, and colours nodes by comparison with the entry. It validates the exported 2/4/12 Add counts and 5/6/7 Math-class counts, and writes these checks to `evidence.json`. No conceptual composition edge is inserted into the native e-graph.

```
make -C docs/papers native-egraph
make -C docs/papers VERSION=v0.2.2
```

A.2 Composition regressions

The paper uses the revised composition terminology. Legacy implementation identifiers remain unchanged: `CoarseRuleComposition` and `SmoothRuleComposition` label context-relative application forms; `SaturatedRuleComposition` stores one exported local state; it is not a trigger-bearing macro or a replacement for the tier0 e-graph. `Saturated` is the serialized status corresponding to a bounded isolated ripen result. The colored examples are checked by `cargo test --release --test paper_math_examples`. Their complete scoped programs are in `docs/papers/examples/`.

```
# Native finite-property regressions
cargo test --release --test closure_contracts --test cs_composition \
  --test saturated_rule_composition --test table_support --test
  saturated_rule_composition_pipeline \
  --test layer_rounds --test ripen --test native_history --test native_debug

# Use a fresh output directory
EGG_LAYOUT_RIPEN_JOBS=128 EGG_LAYOUT_RIPEN_PER_BOUNDARY=32 \
EGG_LAYOUT_RIPEN_MILLISECONDS=1000000 \
cargo run --release --analyze --recapture-tier0 \
  --source experiments/closed_storage/recursive-cs.egg \
  --output out/paper-recursive-cs
```

Evidence	Supported statement	Not implied
Committed event	A recorded effect occurred	Every binding can execute it
Finite witness	A transfer fits observed cases	Arbitrary-count stability
Local saturation	Fixed-entry local execution stabilized	Every embedding is saturated
Exact state match	Declared contents can be shared	Triggers are interchangeable
Ranking proof	Termination under its supply contract	Termination under all injections
Symbolic query contract	Coverage of specified observations	No observer needs intermediates

Table 4: Evidence levels used throughout the report.